

SOC Mid-Term Report

Competitive Programming

Project Title: Competitive Programming

Codeforces Handle: prajwaljais72

Learning Reflection

The first half of the SOC project helped me understand competitive programming in a more structured way. Earlier, I used to look at problems mostly as individual tasks. During these contests, I started noticing that many problems are based on recurring patterns such as greedy choices, sorting-based observations, binary search, graph traversal, and component-based reasoning.

One major realization was that knowing an algorithm is not enough. A large part of competitive programming is about understanding the statement properly, using constraints to guess the required complexity, finding the key observation, and then implementing the idea without mistakes.

Change in My Problem-Solving Approach

During the project, my approach to problems became more systematic. Earlier, I often tried to directly think of a solution or start coding quickly. Now, I try to first understand the brute force approach and why it is too slow. After that, I look at the constraints and try to identify what kind of optimization may be needed.

For many problems, this process helped me understand whether I should think in terms of sorting, greedy choices, binary search, prefix or suffix information, graph traversal, or some other structure.

1. Understand the statement carefully.
2. Check the constraints.
3. Think of a brute force solution.
4. Identify why brute force fails.
5. Search for a useful observation or pattern.
6. Check the idea on small examples.
7. Implement carefully.
8. Test edge cases.

Important Learnings

Reading Constraints Properly

I learned that constraints often give strong hints about the expected solution. If the input size is small, brute force or simulation may be possible. If the input size is large, then an efficient solution such as $O(n)$, $O(n \log n)$, or preprocessing is usually needed.

This helped me avoid wasting time on approaches that could not pass within the time limit.

Greedy Reasoning

Greedy problems were one of the most important parts of my learning. I realized that greedy is not just about choosing what looks best at the moment. The choice must have some reason behind it.

I started asking whether the choice is safe, whether it can make the future worse, whether there is an exchange argument, and whether an invariant is being maintained.

Sorting as a Tool

I learned that sorting is often useful because it reveals structure in the input. After sorting, some problems become easier because elements can be compared, grouped, paired, or processed in a useful order.

This changed how I think about sorting. Earlier, I saw it mostly as a basic operation. Now, I understand that sorting can be a major observation in many problems.

Binary Search on Answer

Binary search on answer was another important idea. I learned that binary search can be used when we are not searching inside an array, but searching for the minimum or maximum possible answer.

The important part is defining a correct check function and ensuring that it is monotonic. This helped me see binary search as a general technique for optimization problems.

Graph Thinking

Graph problems initially felt difficult because they require thinking in terms of nodes, edges, components, and traversal. Through practice, I understood that many graph problems are based on standard ideas like DFS, BFS, visited arrays, and connected components.

I also learned that in disconnected graphs, running DFS or BFS from only one node is not enough. We often need to iterate over all nodes and start a traversal whenever we find an unvisited node.

DSU and Component Reasoning

DSU helped me understand another way of thinking about connectivity. Instead of traversing the graph every time, DSU allows components to be merged efficiently.

The idea of each node having a representative and merging components using union operations was new but useful. I also learned why path compression and union by size or rank are important for efficiency.

Tree DFS

Tree problems helped me understand recursive graph traversal better. Since a tree has no cycles, DFS becomes natural, but it is still important to pass the parent node to avoid going back.

I learned to think about parent-child relationships, subtree processing, and how information can be collected from children during DFS.

Mistakes I Noticed

During the contests and upsolving, I noticed repeated mistakes in my problem-solving process. The main mistakes were starting implementation before fully understanding the statement, not checking constraints carefully, missing edge cases, assuming a greedy idea without justification, making off-by-one errors, forgetting disconnected graphs, and not marking visited nodes correctly.

Identifying these mistakes was useful because now I can consciously try to avoid them.

Value of Upsolving

Upsolving was one of the most useful parts of the project. During contests, it is not always possible to solve every problem. But after the contest, revisiting unsolved problems helped me understand what I missed.

While upsolving, I tried to first think again without time pressure. Then I looked at hints or editorials only when needed. This helped me understand the key observation more deeply.

Upsolving also helped me realize that many problems are difficult mainly because of one hidden idea. Once that idea is understood, the implementation is often manageable.

Improvement Observed

By the mid-term stage, I noticed improvement in my ability to approach problems more patiently and systematically. I became better at reading statements carefully, using constraints to estimate complexity, thinking from brute force before optimizing, recognizing when sorting or greedy may be useful, understanding basic binary search on answer, implementing DFS and BFS, understanding connected components, using DSU for connectivity, debugging simple implementation mistakes, and learning from editorials through upsolving.

Although I still need much more practice, the project helped me build a clearer foundation.

Challenges Faced

The biggest challenge was finding the key observation in problems. Sometimes the solution used only basic concepts, but the main idea was not obvious.

Greedy and constructive problems were especially challenging because they required proof-like thinking. It was not enough to guess a strategy; I had to understand why it works.

Graph and tree problems were also challenging in the beginning because I was less familiar with recursion, visited arrays, and component handling. With practice, these topics started becoming more understandable.

Plan for Improvement

For the second half of the project, I want to focus on improving both consistency and depth.

- Upsolve more unsolved contest problems.
- Practice more graph and tree problems.
- Revise DFS, BFS, DSU, and tree templates.
- Improve implementation speed.

- Spend more time proving why an approach works.
- Analyze mistakes after each contest.
- Focus on recognizing patterns faster.

Conclusion

The first half of the SOC project helped me move from random practice toward a more structured way of learning competitive programming. I learned that improvement comes from regular contests, careful analysis, and serious upsolving.

The most important change was in my thinking process. I now try to understand brute force first, use constraints to guide optimization, look for patterns, and then implement carefully. In the second half of the project, I aim to continue building on this foundation and become more consistent in solving problems independently.